



東北大學

基于 Android 平台的 GNSS 星历解析 APP 设计与实现

汇报人：张洋

学号：20232411

《GNSS定位新技术》课程作业

2026年5月



自强不息 知行合一



自强不息 知行合一

01

开发环境与算法架构

02

技术选型及代码实现

03

运行演示与结果分析

04

项目总结与未来展望



東北大學

目录

CONTENTS



Kotlin 介绍

Kotlin (科特林) 是由JetBrains公司开发的静态编程语言，推出于2011年7月，支持编译为Java字节码、JavaScript及二进制代码，适用于**Android**、嵌入式设备等多平台开发。



Kotlin 优势

(1) 富有表现力且简洁：

您可以**使用更少的代码实现更多的功能**。表达自己的想法，少编写样板代码。在使用 Kotlin 的专业开发者中，有 67% 的人反映其工作效率有所提高。

(2) 更安全的代码：

Kotlin 有许多语言功能，可帮助您避免 null 指针异常等常见编程错误。包含 Kotlin 代码的 Android 应用**发生崩溃的可能性降低了 20%**。



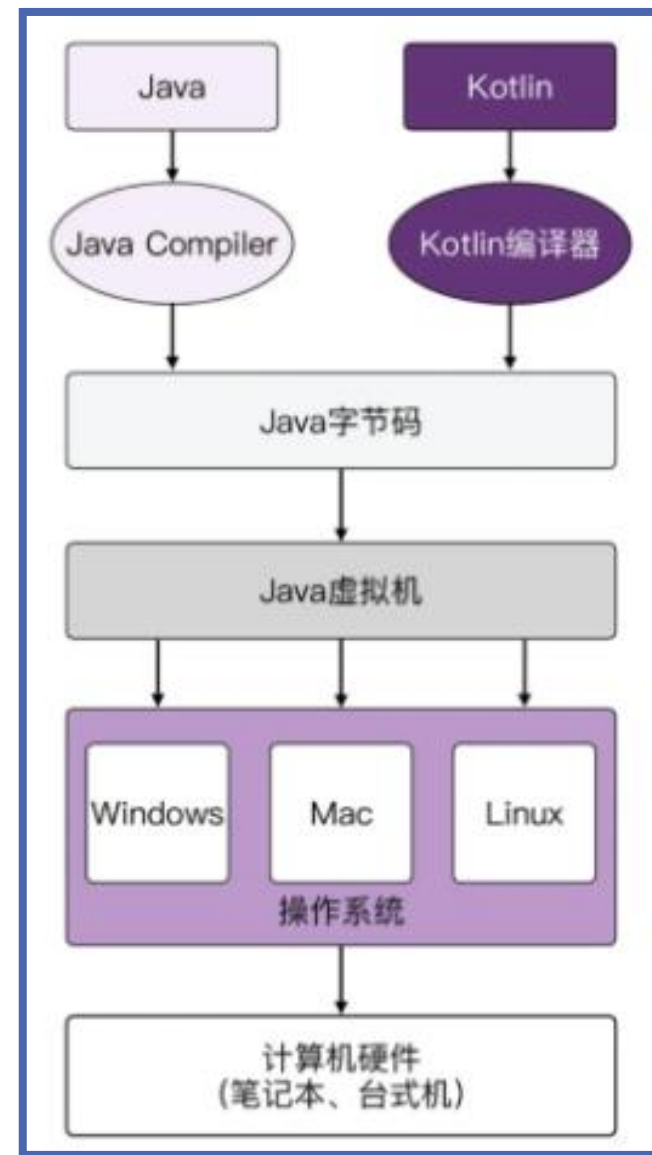
Kotlin 优势

(3) 可互操作:

您可以在 Kotlin 代码中调用 Java 代码，或者在 Java 代码中调用 Kotlin 代码。**Kotlin 可完全与 Java 编程语言互操作**，因此您可以根据需要在项目中添加任意数量的 Kotlin 代码。

(4) 结构化并发:

Kotlin 协程让异步代码像阻塞代码一样易于使用。协程可**大幅简化后台任务管理**，例如网络调用、本地数据访问等任务的管理。





算法架构



代码实现



结果分析

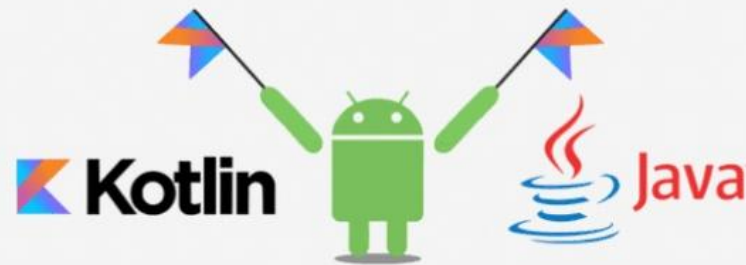


总结展望

Kotlin 与 JAVA 的对比

特性	Kotlin	Java
类型推断	支持带有类型推断的简洁语法。	需要显式的类型声明。
空安全	内置空安全特性，减少空指针异常。	空安全不是固有的；空指针异常很常见。
扩展函数	支持扩展函数，允许开发者在不修改现有类的情况下为其添加新函数。	缺乏扩展函数；新功能通常需要修改现有类。
互操作性	与Java完全互操作。现有的Java代码可以在Kotlin中无缝使用，反之亦然。	虽然Kotlin可以调用Java代码，但Java可能无法直接调用Kotlin代码，需要进行一些适配。
简洁性	通常更简洁，样板代码更少，从而提高开发者的生产力。	语法更冗长，包含额外的样板代码。
开发的应用示例	zomato NETFLIX Uber	Spotify X NASA

Apps written in Kotlin





算法流程图

该算法流程是一个完整的GNSS单点定位处理系统，主要分为五个核心阶段：**数据采集与预处理、卫星位置计算、伪距观测模型与修正、最小二乘定位解算以及结果输出与分析。**

首先，**通过Android GNSS API采集原始观测数据并接收导航电文**，结合接收机近似位置等辅助信息，进行周跳检测、异常值剔除等预处理。随后，**基于广播星历解析卫星轨道参数**，通过开普勒方程迭代求解偏近点角，计算卫星在地心地固坐标系中的精确位置，并修正地球自转和卫星钟差。接着，建立伪距观测模型，计算几何距离并修正电离层、对流层延迟及接收机钟差，通过权重模型筛选有效卫星。然后，**采用加权最小二乘法迭代求解接收机位置**，通过构建设计矩阵和观测残差，利用高斯消元法解算法方程，直至位置更新量小于收敛阈值。最终，**输出定位结果、精度因子、可视化图表及各类数据记录**，完成从原始观测到高精度定位的全流程处理。

数据采集
和预处理



卫星位置
计算



伪距观测
修正



最小二乘
定位解算



结果输出
分析



算法架构



代码实现



结果分析



总结展望

数据采集与预处理

通过Android GNSS API采集**原始观测数据**（伪距、载波相位等），结合**导航电文**（广播星历、电离层参数等）与**辅助信息**（接收机近似位置、时间等），经**周跳检测**、**异常剔除**、**数据同步**等**预处理步骤**，输出可供后续计算的观测数据与辅助信息。





算法架构



代码实现



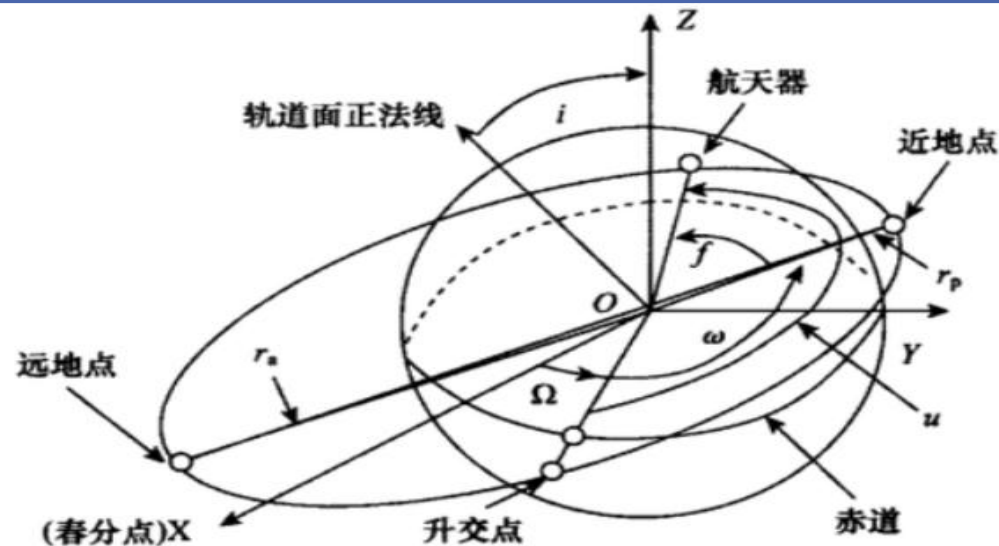
结果分析



总结展望

卫星位置计算

基于广播星历解析，通过平近点角计算、开普勒方程迭代求解偏近点角、真近点角与升交角距计算、二阶谐波修正、轨道平面坐标转换、地球自转修正及卫星钟差修正等步骤，最终输出卫星在ECEF坐标系下的精确位置。



2.1 平近点角计算

半长轴: $A = (\sqrt{A})^2$ · 平均角速度: $n_0 = \sqrt{\frac{\mu}{A^3}}$
修正后角速度: $n = n_0 + \Delta n$

2.2 开普勒方程迭代求解偏近点角

时间差: $t_k = t - t_{oc}$ (周跳越修正)
平近点角: $M_k = M_0 + n \cdot t_k$

2.3 真近点角与升交角距

开普勒方程: $M_k = E_k - e \sin E_k$
迭代: $E_k^{(i+1)} = M_k + e \sin E_k^{(i)}$
收敛条件: $|E_k^{(i+1)} - E_k^{(i)}| < \epsilon$

2.4 二阶谐波修正

真近点角: $v_k = \arctan\left(\frac{\sqrt{1-e^2} \sin E_k}{\cos E_k - e}\right)$
升交角距: $\phi_k = v_k + \omega$

2.5 轨道平面坐标

$\delta u_k = C_{us} \sin 2\phi_k + C_{uc} \cos 2\phi_k$
 $\delta r_k = C_{rs} \sin 2\phi_k + C_{rc} \cos 2\phi_k$
 $\delta i_k = C_{is} \sin 2\phi_k + C_{ic} \cos 2\phi_k$

2.6 ECEF 坐标转换

$u_k = \phi_k + \delta u_k$
 $r_k = A(1 - e \cos E_k) + \delta r_k$
 $i_k = i_0 + i' t'_k + \delta i_k$
 $x'_k = r_k \cos u_k$, $y'_{ok} = r_k \sin u_k$

2.7 地球自转修正 (Sagnac 效应)

$\Omega_k = \Omega_0 + (\dot{\Omega} - \omega_e)t_k - \omega_e t_{oc}$
 $\begin{cases} X_s = x'_k \cos \Omega_k - y'_{ok} \sin \Omega_k \\ Y_s = x'_k \sin \Omega_k + y'_{ok} \cos \Omega_k \\ Z_s = y'_{ok} \sin i_k \end{cases}$

2.8 卫星钟差修正

$X_s^{rot} = X_s \cos(\omega_e \tau) + Y_s \sin(\omega_e \tau)$
 $Y_s^{rot} = -X_s \sin(\omega_e \tau) + Y_s \cos(\omega_e \tau)$
 $Z_s^{rot} = Z_s$, $\tau = r_k/c$

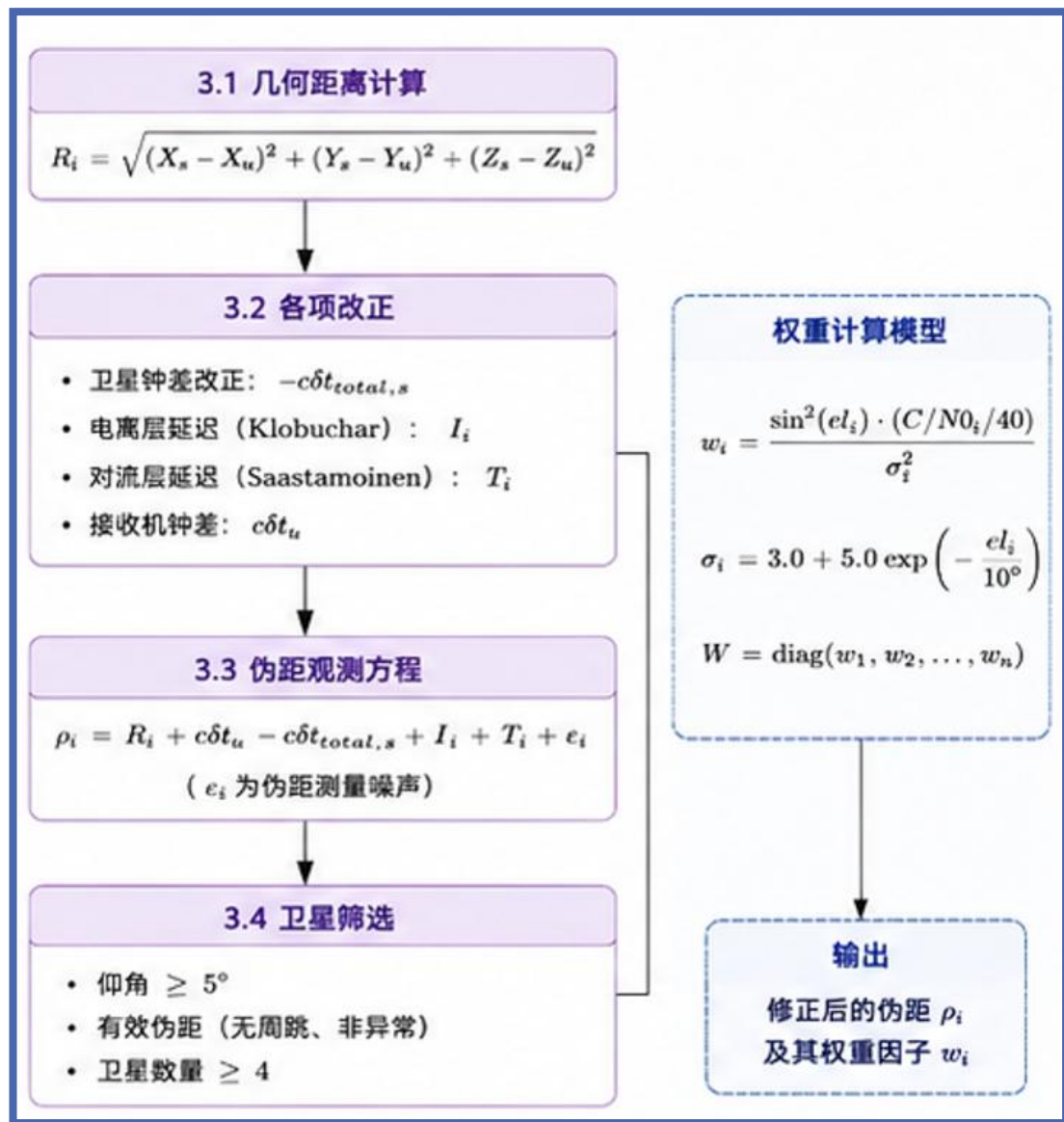
2.9 输出结果

卫星钟差: $\delta t_s = a_{f0} + a_{f1}(t - t_{oc}) + a_{f2}(t - t_{oc})^2$
相对论修正: $\Delta t_{rel} = -\frac{2\sqrt{\mu}}{c^2} e \sqrt{A} \sin E_k$
总钟差: $\delta t_{total,s} = c(\delta t_s + \Delta t_{rel})$
输出: 卫星 ECEF 坐标 (X_s, Y_s, Z_s)
卫星钟差 $\delta t_{total,s}$



伪距观测模型与修正

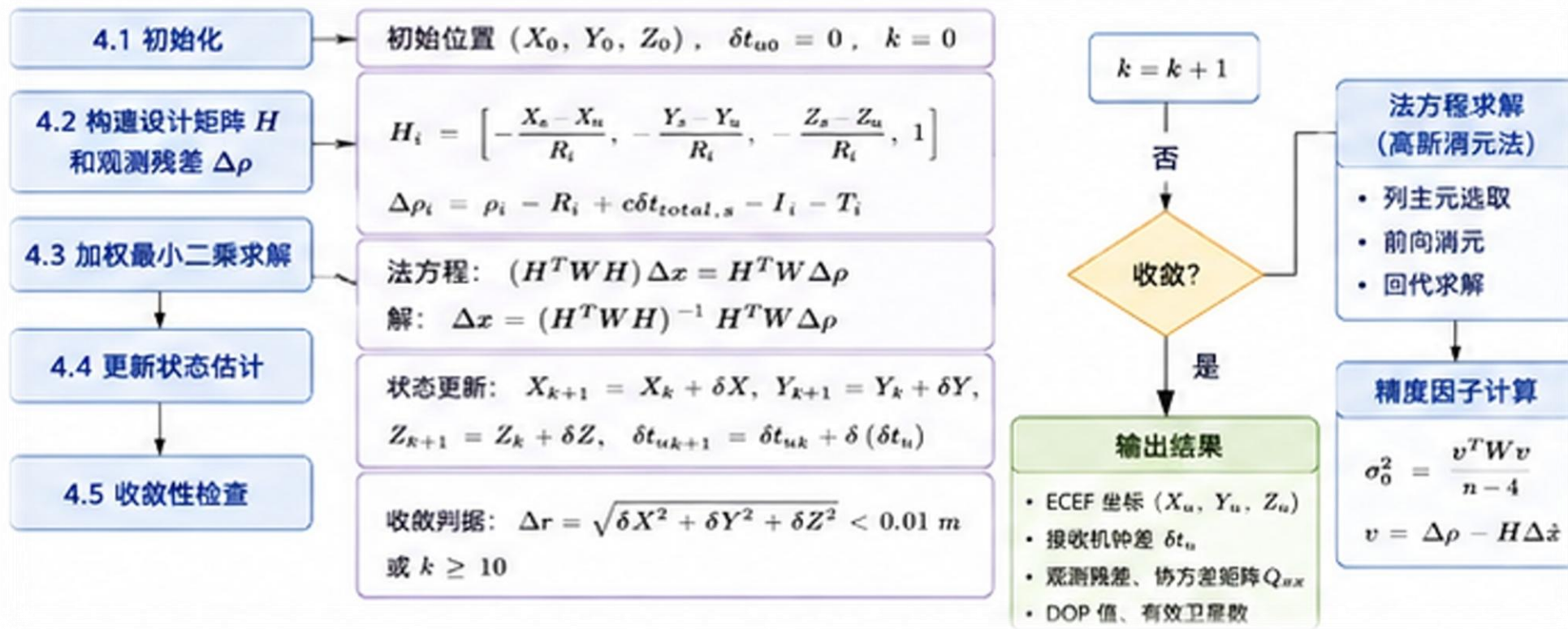
先计算卫星与接收机的几何距离，再通过**卫星钟差**、**电离层延迟**（Klobuchar模型）、**对流层延迟**（Saastamoinen模型）、**接收机钟差**等修正项优化伪距观测值，结合**权重计算模型**与**卫星筛选**（仰角、有效伪距、卫星数量）条件，输出修正后的伪距及权重因子。





最小二乘定位解算 (SPP)

初始化接收机位置与钟差后，**构建设计矩阵与观测残差**，通过**加权最小二乘法**求解状态更新量，**迭代检查收敛性**（位置变化或迭代次数），最终输出ECEF坐标、LLA坐标、接收机钟差及精度因子等定位结果。





结果输出与分析

包含定位结果（坐标、速度等）、精度评估（HDOP/VDOP等、卫星数量分布）、可视化展示（天空图、轨迹）、数据记录（CSV/TXT/LOG文件）及质量分析（误差统计、收敛时间、可用性），全面呈现定位性能与数据特征。

5.1 定位结果



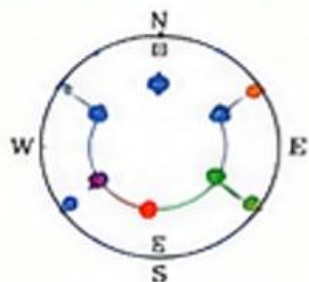
- ECEF 坐标
- LLA 坐标
(经纬度高程)
- 速度 (可选)
- 定位状态

5.2 精度评估

- HDOP / VDOP / PDOP
- GDOP / TDOP
- 卫星数量
- 几何分布



5.3 可视化展示



- 天空图 (Sky Plot)
- 信号强度柱状图
- 定位状态指示
(固定/浮点/单点)
- 轨迹显示 (可选)

5.4 数据记录

- 观测数据 (CSV)
- 定位结果 (CSV)
- 系统日志 (TXT)
- 原始电文 (LOG)



5.5 质量分析

- 误差统计 (RMS)
- 收敛时间分析
- 可用性分析
- 性能评估





MVVM模式与分层架构

应用通过状态去渲染更新UI是程序设计中相对复杂，但又十分重要的，往往决定了应用程序的性能。程序的状态数据通常包含了数组、对象，或者是嵌套对象组合而成。

在这些情况下，采取 **MVVM = Model + View + ViewModel** 模式，其中状态管理模块起到的就是ViewModel的作用，将数据与视图绑定在一起，更新数据的时候直接更新视图。





View 层（界面展示与用户交互）

文件	类型	职责
MainActivity.kt	Activity	权限申请、导航设置、WindowInsets 适配
PositionFragment.kt	Fragment	定位页面 UI 更新
SkyPlotFragment.kt	Fragment	天空图页面，管理卫星列表显示
QualityFragment.kt	Fragment	质量分析页面（DOP、信噪比统计）
SecurityFragment.kt	Fragment	信号完整性检测页面
LogFragment.kt	Fragment	日志记录控制与原始数据展示
SkyPlotView.kt	自定义 View	绘制极坐标卫星天空图
fragment_*.xml 等	布局文件	定义各页面卡片式布局
colors.xml, themes.xml	资源	微信风格配色与主题



算法架构



代码实现



结果分析



总结展望

ViewModel 层（业务逻辑与数据持有）

文件	类型	职责
MainViewModel.kt	ViewModel (AndroidViewModel)	协调所有 Manager、定位引擎、日志记录； 通过 LiveData 暴露数据给 View

Model 层（数据、算法、仓储、工具）

数据模型（data class）

文件	说明
RawMeasurement.kt	原始观测数据（伪距、载噪比、载波相位等）
SatelliteInfo.kt	卫星状态（仰角、方位角、是否用于定位）
NavigationData.kt	广播星历（GpsEphemeris、GlonassEphemeris、电离层参数）
PositionResult.kt	定位结果（LLA、DOP、精度估计等）



算法架构



代码实现



结果分析



总结展望

数据仓库 (Repository)

文件	说明
GnssRepository.kt	集中管理所有 LiveData, 作为单一数据源

数据源 (GNSS 采集与解析)

文件	说明
GnssMeasurementManager.kt	注册 GnssMeasurementsEvent.Callback, 计算伪距
GnssNavigationManager.kt	注册导航消息回调, 解析 GPS 广播星历
GnssStatusManager.kt	注册卫星状态回调, 提供可见卫星列表
GnssClockHandler.kt	解析 GNSS 时钟信息, 计算 GPS 时间
EphemerisDownloader.kt	从 A-GPS 服务器下载 XTRA 辅助数据



算法架构



代码实现



结果分析



总结展望

定位算法

文件	说明
SppEngine.kt	单点定位引擎（星历定位 + 加权质心法降级）
SatellitePositionCalc.kt	计算卫星 ECEF 坐标、仰角、方位角；电离层/对流层延迟模型
LeastSquaresSolver.kt	加权最小二乘解算、DOP 计算、高斯消元法

日志与工具

文件	说明
CsvLogger.kt	CSV 格式日志写入（兼容 Google GnssLogger）
LogFileManager.kt	日志文件管理（列表、删除、导出）
Constants.kt	GNSS 物理常量、星座类型、系统参数
CoordinateUtils.kt	ECEF/LLA/ENU 坐标转换、Haversine 距离
TimeUtils.kt	GPS 时间与 Unix 时间互转



伪距计算

Android 底层芯片提供的是原始时间计数需要**通过该函数转换为以米为单位的伪距**。代码中首先将接收机本地时钟修正为 GPS 系统时间，得到接收时刻的 GPS 时间纳秒数。接着，**获取卫星发射，两者必须通过取模运算进行周期对齐**。对齐后的时间差再乘以光速，即得到伪距。

该函数是系统数据采集层的核心，其计算结果直接决定了后续所有定位解算的质量。

```
private fun calculatePseudorange(
    measurement: GnssMeasurement,
    clock: GnssClock
): Double {
    if (!clock.hasFullBiasNanos()) return 0.0
    val fullBias = clock.fullBiasNanos
    val bias = if (clock.hasBiasNanos()) clock.biasNanos else 0.0
    val timeNanos = clock.timeNanos
    val rxTimeNanos = timeNanos - fullBias - bias
    val svTime = measurement.receivedSvTimeNanos
    val timeOffset = measurement.timeOffsetNanos
    val periodNanos = when (measurement.constellationType) {
        CONSTELLATION_GLO_NASS -> (86400.0 * 1e9).toLong()
        else -> (GPS_WEEK_SECONDS * 1e9).toLong()
    }
    var pseudorangeNanos = (rxTimeNanos - svTime - timeOffset) % periodNanos
    if (pseudorangeNanos < 0) pseudorangeNanos += periodNanos
    return pseudorangeNanos * SPEED_OF_LIGHT / 1e9
}
```



开普勒方程迭代求解偏近点角

该代码段的关键在于求解开普勒方程 $M_k = E_k - e \cdot \sin(E_k)$ 。由于该方程为超越方程，无法解析求解，系统采用逐次逼近迭代法，通常 5 ~ 8 次即可收敛到 $1e-14$ 弧度的精度。得到 E_k 后，再利用几何关系计算真近点角 v_k 。真近点角是卫星在轨道平面内相对于近地点的真实角度，是后续升交角距、轨道半径、ECEF 坐标计算的关键中间量。

```
var ek = mk
for (i in 0 until 10) {
    val ekNew = mk + ephemeris.eccentricity * sin(ek)
    if (abs(ekNew - ek) < 1e-14) break
    ek = ekNew
}
val sinVk = sqrt(1 - e * e) * sin(ek) / (1 - e * cos(ek))
val cosVk = (cos(ek) - e) / (1 - e * cos(ek))
val vk = atan2(sinVk, cosVk)
```




加权最小二乘定位解算

```
fun solve(  
    satellites: List<SatelliteData>,  
    initialPosition: Triple<Double, Double, Double>  
) : Solution? {  
    var (x, y, z) = initialPosition  
    var clockBias = 0.0  
    for (iter in 0 until MAX_ITERATIONS) {  
        val h = Array(n) { DoubleArray(4) }  
        val deltaRho = DoubleArray(n)  
        for (i in satellites.indices) {  
            val sat = satellites[i]  
            val range = sqrt((sat.x - x)^2 + (sat.y - y)^2 + (sat.z - z)^2)  
            h[i][0] = -(sat.x - x) / range  
            h[i][1] = -(sat.y - y) / range  
            h[i][2] = -(sat.z - z) / range  
            h[i][3] = 1.0  
            deltaRho[i] = sat.pseudorange - (range + clockBias - sat.clockBiasMeters)  
        }  
        val solution = solveWeightedLeastSquares(h, deltaRho, weights)  
        x += solution[0]; y += solution[1]; z += solution[2]; clockBias += solution[3]  
        if (sqrt(solution[0]^2 + solution[1]^2 + solution[2]^2) < 0.01) break  
    }  
    return Solution(x, y, z, clockBias, ...)  
}
```

伪距单点定位（SPP）的核心是一个非线性最小二乘问题，未知数为接收机的三维坐标（X, Y, Z）和接收机钟差，共 4 个未知数。首先根据当前估计的接收机位置计算各卫星的几何；然后构建设计矩阵 H；同时计算观测残差。加权最小二乘解法考虑了卫星仰角、信噪比等因素，能够更合理地分配各卫星的权重。每次迭代得到位置和钟差的改正量，当改正量小于 0.01 米时认为收敛。



定位协调与数据流

```
measurementManager?.register(object : OnMeasurementsReceivedListener {  
    override fun onMeasurementsReceived(  
        measurements: List<RawMeasurement>,  
        clockInfo: GnssClockInfo  
    ) {  
        repository.updateMeasurements(measurements)  
        if (csvLogger.isLoggingActive()) {  
            csvLogger.logMeasurements(measurements, clockInfo)  
        }  
        viewModelScope.launch(Dispatchers.Default) {  
            trySolvePosition(measurements, clockInfo)  
        }  
    }  
})
```

该代码段体现了系统的 **MVVM 架构** 和 **协程异步数据流设计**。当 Android 底层 GNSS 芯片产生新的测量数据时，Gns sMeasurementManager 通过回调将转换后的 RawMeasurement 列表和时钟信息传递给 MainViewModel。View Model 首先将这些数据通过 Repository 更新到 LiveData 中，使得 **UI 层可以被动观察并刷新界面**。最后，在 Dispatchers.Default 线程池中启动协程执行定位解算 trySolvePosition，**避免阻塞主线程**。



伪距观测方程残差构建

这两行代码虽然简短，却是整个单点定位数学模型的**直接体现**。根据伪距观测方程：

$$\rho = r + c \cdot \delta t_u - c \cdot \delta t_s + I + T + \varepsilon$$

在忽略电离层、对流层和多路径（或已通过模型修正）的情况下，预测的伪距值应当为几何距离 `range` 加上接收机钟差 `clockBias` 再减去卫星钟差 `sat.clockBiasMeters`。**实际测量的伪距与这个预测值之间的差即为观测残差 `deltaRho`**。该残差被送入最小二乘求解器，驱动接收机位置和钟差的迭代修正。正是通过反复调整接收机位置，**使得所有卫星的残差平方和最小化，最终收敛到最优**的定位结果。因此，这两行代码连接了物理测量值与数学估计值，是实现“从伪距到位置”这一核心转换的关键纽带。

```
val predictedRange = range + clockBias - sat.clockBiasMeters
deltaRho[i] = sat.pseudorange - predictedRange
```



运行软硬件要求

该 Android GNSS 星历解析 APP 的运行环境和条件主要包括软件平台、系统权限、硬件支持以及网络要求四个方面。

在**软件环境**方面，应用**基于 Android 8.0 (API 26) 及以上版本**开发，目标 SDK 为 API 35，使用 Kotlin 2.0.21 语言编写，通过 Android Studio 2024.x 及 Gradle 9.4.1 构建，运行时需要 JDK 17 环境。应用必须获得 **ACCESS_FINE_LOCATION 精确位置权限**（建议同时授予 ACCESS_COARSE_LOCATION），以便通过 Android 的 GnssMeasurement API 获取原始观测量和广播星历。

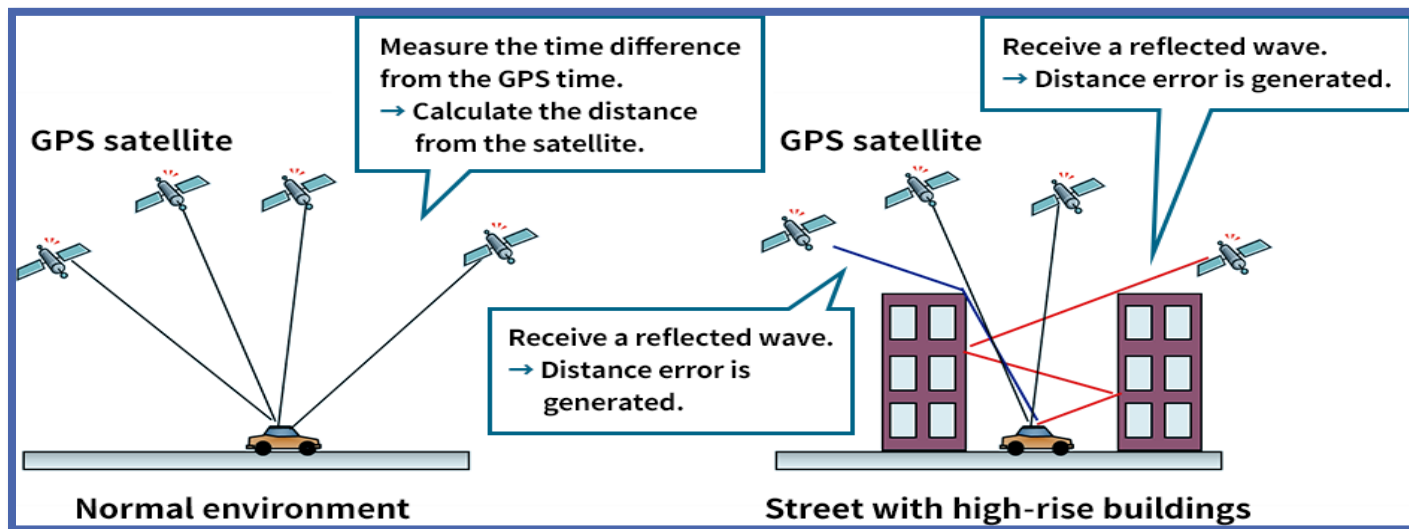
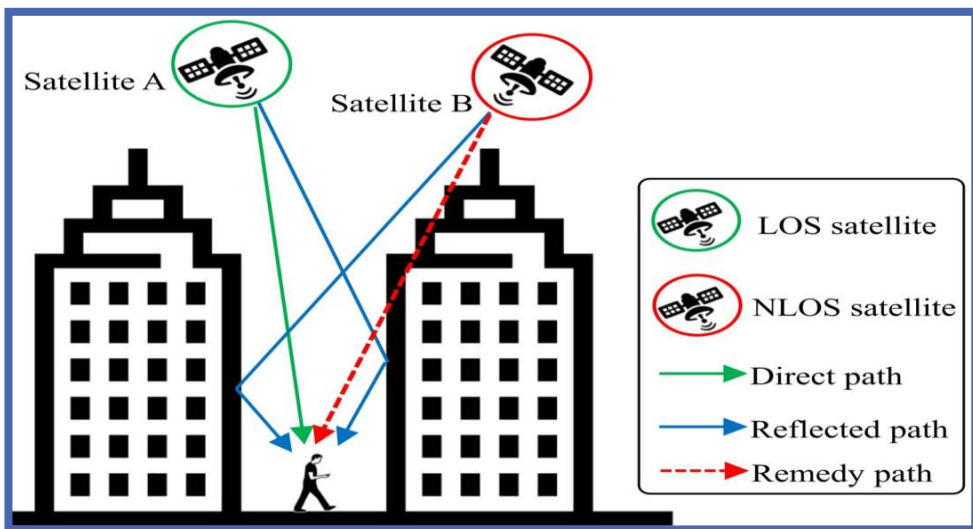
在**硬件层面**，设备需配备**支持原始 GNSS 观测值输出的芯片**（自 Android 7.0 起多数中高端手机支持，如 Pixel、三星、华为、小米等品牌机型），并建议**支持多星座（GPS、BDS、GLONASS、Galileo）以提升定位性能**；双频（L1/L5 或 E1/E5a）支持并非必须，但可改善电离层修正效果。



使用注意事项

应在**开阔环境中使用应用，确保天线有良好的天空视野**。混凝土建筑内信号衰减严重，会导致定位困难甚至无法定位。恶劣天气（如强降雨、大风雪等）会导致信号质量下降、精度降低。应避免在高速运动中使用应用，因为该应用设计针对静止或低速场景。

GNSS 接收和数据计算耗电量较大，长时间运行会导致设备发热。**建议采集数据时连接充电器或移动电源。**





定位页面

定位页面在设计上采用了渐进式信息展示策略：在初始状态下，仅显示必要的状态提示和操作按钮；在定位成功后，**逐步展示坐标信息、精度指标和卫星概览**。这种设计减少了用户的信息过载，使界面更加清晰友好。

左侧为初始等待定位状态，显示红色“等待定位...”提示，坐标信息和精度指标均为空值。右侧为已定位状态，系统成功获取坐标，精度达到 4.0 米。顶部状态栏实时显示可见卫星数和测量计数。





算法架构



代码实现



结果分析



总结展望

天空图页面

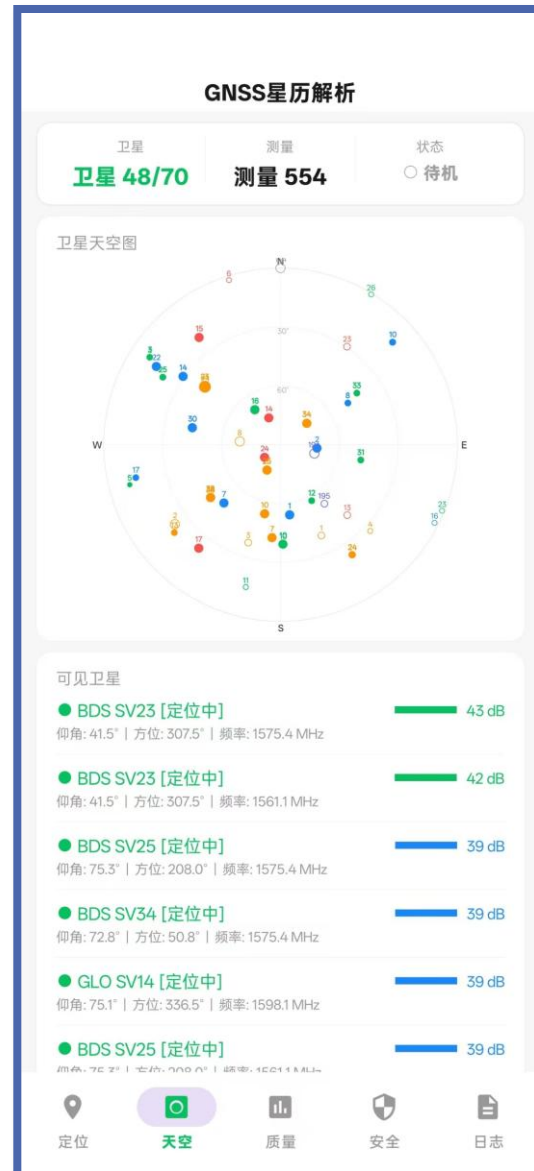
天空图页面以极坐标投影直观展示所有可见卫星的空间分布。将复杂的卫星空间配置转换为**直观的极坐标可视化**，便于用户评估几何分布。

极坐标系统：圆心对应天顶，外圆对应地平线。同心圆表示 60度和30°度仰角圈，帮助用户快速判断卫星的高度。

星座配色：不同星座采用不同颜色——BDS 橙色、GPS 蓝色、Galileo 绿色、GLONASS红色、QZSS 紫色。颜色区分降低了认知负担。

卫星符号：实心圆表示参与定位的卫星，空心圆表示仅可见。圆点大小与 C/N0 成正比。

卫星列表：下方列表展示每颗卫星的详细参数（星座类型、SVID、仰角、方位角、频率、C/N0），以进度条可视化信噪比强度。





算法架构



代码实现



结果分析



总结展望

质量分析页面

质量分析页面将繁多的统计数据组织为三个独立的功能区域：

DOP 指标区域：以彩色进度条展示 GDOP=2.2、PDOP=1.8、HDOP=1.2、VDOP=1.4，综合评价“优秀”。

信噪比统计区域：显示平均 C/N0=26 dB-Hz、最大 41 dB-Hz、最小 0 dB-Hz。信号强度分为三档：强信号 3 颗、中等 25 颗、弱 42 颗。

星座统计区域：按 BDS（23 颗）、GAL（20 颗）、GPS（15 颗）、GLO（7 颗）、QZS（4 颗）、SBAS（1 颗）分组统计，显示各系统的平均信噪比和参与定位的卫星数。





算法架构



代码实现



结果分析



总结展望

安全检测页面

安全检测页面提供全面的信号完整性评估：

顶部显示“轻微异常”状态；

中级展示三项异常指标（伪距残差 10.1 dB、载波跳变 70 颗、多径指数 0 颗）；

底部列出五项检测项目的逐项评定（信号强度一致性“正常”、卫星数量合理性“异常”、低仰角信号“正常”、星座完整性“正常”、频率一致性“警告”）。

安全检测页面用于评估定位的误差来源，定位条件是否良好，在实际使用时可以调整至完整状态提升定位精度。





算法架构



代码实现



结果分析



总结展望

日志记录页面

日志记录界面，顶部提供“停止记录”和“清除日志”按钮，中部列表实时显示卫星原始观测量（SVID、星座、C/N0、PR、CP、频率），底部显示文件名和“导出 CSV”按钮。

下图为 CSV 文件在 Excel 中的视图。

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
1	#	Raw	GNSS	Measurements	Log																		
2	Version	2.41																					
3	Applicatio	GNSSLogger																					
4	DeviceMo	PHK110																					
5	AndroidVe	16																					
6	StartTime	1.78E+12																					
7																							
8	Fix	Provider	Latitude	Longitude	Altitude	Speed	Bearing	Accuracy	UnixTime	Millis													
9	Raw	TimeNanc	TimeOffse	ReceivedS	ReceivedS	State	Svid	Constellat	Cn0DbHz	Pseudorar	Pseudorar	Accumula	Accumula	Accumula	CarrierFre	CodeType	Multipath	SnrInDb	Baseband	FullInterSi	SatelliteIn	Pseudorange	Meters
10	Clock	TimeNanc	FullBiasNz	BiasNanos	BiasUncer	DriftNano	DriftUncer	Hardware	LeapSecond														
11	Clock	2.03E+14	-1.5E+18	-0.48735	39.44386	-5.67797	10.42469	100	18														
12	Raw	2.03E+14	0	3.12E+14	53	16399	10	1	23.5	428.7982	0.9165	16	0	0	1.58E+09	C	0	0	19.9	0	0	2.14E+07	
13	Raw	2.03E+14	0	3.12E+14	27	16399	12	1	28	310.4546	0.618	16	0	0	1.58E+09	C	0	0	24.4	0	0	2.19E+07	
14	Raw	2.03E+14	0	3.12E+14	70	16399	24	1	21.1	657.1602	1.093	16	0	0	1.58E+09	C	0	0	17.5	0	0	2.49E+07	
15	Raw	2.03E+14	0	3.12E+14	17	16399	25	1	32.5	-157.062	0.056294	16	0	0	1.58E+09	C	0	0	28.9	0	0	2.03E+07	
16	Raw	2.03E+14	0	3.12E+14	13	16399	28	1	35.1	-367.74	0.026392	16	0	0	1.58E+09	C	0	0	31.5	0	0	2.17E+07	
17	Raw	2.03E+14	0	3.12E+14	6	16399	32	1	41.7	-217.559	0.045806	16	0	0	1.58E+09	C	0	0	38.1	0	0	2.07E+07	
18	Raw	2.03E+14	0	3.12E+14	16	16399	194	4	33.4	-35.3871	0.053405	16	0	0	1.58E+09	C	0	0	29.8	0	0	3.80E+07	
19	Raw	2.03E+14	0	6.34E+13	11	49359	6	3	37.5	318.2184	0.045596	16	0	0	1.60E+09	C	0	0	33.5	3795.105	0	0	
20	Raw	2.03E+14	0	6.34E+13	24	49359	9	3	29.1	-375.432	0.112295	16	0	0	1.60E+09	C	0	0	25.1	3795.105	0	0	
21	Raw	2.03E+14	0	6.34E+13	10	49359	16	3	38.8	426.171	0.044803	16	0	0	1.60E+09	C	0	0	34.8	3795.105	0	0	
22	Raw	2.03E+14	0	6.34E+13	7	49359	7	3	41.1	-476.718	0.044628	16	0	0	1.60E+09	C	0	0	37.1	3795.105	0	0	
23	Raw	2.03E+14	0	3.12E+14	19	16399	42	5	31.4	-118.683	0.045445	16	0	0	1.56E+09	I	0	0	27.1	-1540.74	0	0	
24	Raw	2.03E+14	0	3.12E+14	10	16399	39	5	38.9	471.6907	0.034453	16	0	0	1.56E+09	I	0	0	34.6	-1540.74	0	0	
25	Raw	2.03E+14	0	3.12E+14	19	16399	33	5	31.3	-572.926	0.061945	16	0	0	1.56E+09	I	0	0	27	-1540.74	0	0	
26	Raw	2.03E+14	0	3.12E+14	12	16399	26	5	37.2	-113.078	0.025566	16	0	0	1.56E+09	I	0	0	32.9	-1540.74	0	0	
27	Raw	2.03E+14	0	3.12E+14	46	16399	24	5	25.1	-540.793	0.8	16	0	0	1.56E+09	I	0	0	20.8	-1540.74	0	0	
28	Raw	2.03E+14	0	3.12E+14	40	16399	21	5	25.8	422.6597	0.794	16	0	0	1.56E+09	I	0	0	21.5	-1540.74	0	0	
29	Raw	2.03E+14	0	3.12E+14	12	16399	14	5	36.7	178.8814	0.025566	16	0	0	1.56E+09	I	0	0	32.4	-1540.74	0	0	



常见故障排查

1. 无法获取位置信息?

若应用启动后卫星列表始终为空或显示的卫星数为 0，首先**确认位置权限已授予**（通过Settings > Apps > GNSS 星历解析 APP > Permissions 验证）。**在开阔环境中使用应用**，避免室内或有遮挡的地方。关闭应用后彻底关闭系统定位服务，等待 30 秒后重新打开应用。此外应**检查设备 GNSS 芯片是否正常**（在系统设置中查看 GNSS 状态），并尝试**在晴朗天气下使用**，因为恶劣天气会严重影响 GNSS 信号。

2. 数据文件无法导出?

若停止记录后未生成 CSV 文件，首先**确认存储空间充足**（至少 100MB 可用）。检查是否已**授予外部存储写入权限**。尝试使用文件管理器检查相关目录的权限设置。通常这些步骤能够解决文件导出问题。



算法架构



代码实现



结果分析



总结展望

项目总结

本次基于 Android 平台设计并实现了一套完整的 GNSS 星历解析与单点定位系统。系统利用 Android 7.0 以来开放的原始 GNSS 观测量 API，采用 Kotlin 语言和 MVVM 架构，实现了从原始数据采集、广播星历解析、卫星位置计算（开普勒方程迭代求解）、加权最小二乘定位解算，到 Klobuchar 电离层修正、Saastamoinen 对流层修正、DOP 精度因子计算以及坐标系统转换（EC EF/LLA/ENU）的全链路自主定位处理。提供了卫星天空图极坐标可视化、多维度信号质量分析、信号完整性检测以及 CSV 格式原始数据记录等辅助功能。

通过实际在 Android 设备上的测试验证，系统能够在开阔环境下成功实现实时多星座 GNSS 数据处理，伪距单点定位（SPP）水平精度可达 5 米以内，HDOP 值通常处于 0.6~1.5 的优秀区间，证明了在移动智能终端上进行底层 GNSS 自主数据处理的可行性。同时，项目在软件工程方面也取得了显著成果：采用 MVVM 架构实现关注点分离，通过 LiveData 实现数据驱动的 UI 自动更新，使用 Kotlin 协程处理异步任务，保证了系统的可维护性和扩展性。



未来展望

尽管当前系统已经实现了基本功能，但仍存在若干不足，也为未来的技术演进指明了方向。

首先，定位精度受限于伪距噪声和简化的大气模型，**复杂环境下多路径效应会导致误差显著增大**。其次，逐历元独立求解导致定位结果跳跃明显，**缺少实时滤波和平滑机制**。

未来，系统可以从以下几个方面进行深度优化与扩展：

一是**引入载波相位平滑伪距技术**（Hatch 滤波器），利用高精度的载波相位观测值将伪距噪声从米级降低至分米级；

二是**集成扩展卡尔曼滤波**（EKF）或粒子滤波，对位置、速度、钟差进行状态估计，提升定位的连续性和稳定性；

三是**增强大气延迟修正模型**，引入 GPT3 或数值天气预报产品，提高湿延迟和电离层活跃期的修正精度。



東北大學

汇报完毕，请老师批评指正！

THAT'S ALL FOR MY PRESENTATION. THANK YOU!

汇报人：张洋

学号：20232411

《GNSS定位新技术》课程作业

2026年5月



自强不息 知行合一